Assignment 4: Sequential Modeling with Recurrent Networks

Course: Large Learning Models

Assignment Report

Submitted by:

Akhil Agrawal
Bibaswan Das
Srijan Singh

Group: 9

Submitted on: 12 October 2026

Contents

• Overview

• TASK 1: Kannada Digit Classification with RNN, LSTM and GRU

• 1. Objectives

• 2. Dataset and Preprocessing

• 3. Methodology

• 4. Experimental Results and Analysis

• 5. Task 1 Conclusion

• TASK 2: Spoken Arabic Digit Recognition with Recurrent Networks

• 1. Objectives

• 2. Dataset and Preprocessing

• 3. Methodology

• 4. Experimental Results and Analysis

• 5. Limitations and Interpretation

• 6. Task 2 Conclusion

• Overall Conclusion

Overview

This report documents two sequential-classification experiments implemented in PyTorch. Task 1 converts Kannada handwritten digit images into sequences of image rows. Task 2 classifies spoken Arabic digits from variable-length sequences of 13-dimensional MFCCs. The notebooks compare RNN, LSTM, and GRU models across 36 configurations for Task 1 and 18 for Task 2.

Both notebooks use Adam and cross-entropy loss and stop when the absolute difference between consecutive average training losses is below 10^{-4}. Neither notebook uses validation accuracy for model selection. Both rank configurations by test accuracy. Task 2's best model reaches 100.00% training accuracy but only 12.18% test accuracy; this result is reported as observed and discussed cautiously.

TASK 1: Kannada Digit Classification with RNN, LSTM and GRU

1. Objectives

Classify ten Kannada digit classes by treating each 28 × 28 grayscale image as 28 time steps, each with 28 pixel features. Compare cell types, hidden sizes, and layer counts using training/test accuracy, loss curves, confusion matrix, and sample visualizations.

2. Dataset and Preprocessing

The notebook reads train.csv and reports 42,000 training, 9,000 validation, and 9,000 test samples. Splits use random_state=42 but are not stratified. Pixel values are reshaped to 28 × 28 and divided by 255, giving inputs of shape (batch, 28, 28). The validation loader is created but not used in training or model selection.

Subset	Samples	Role
Training	42,000	Gradient updates
Validation	9,000	Created but not used
Test	9,000	Reported test accuracy

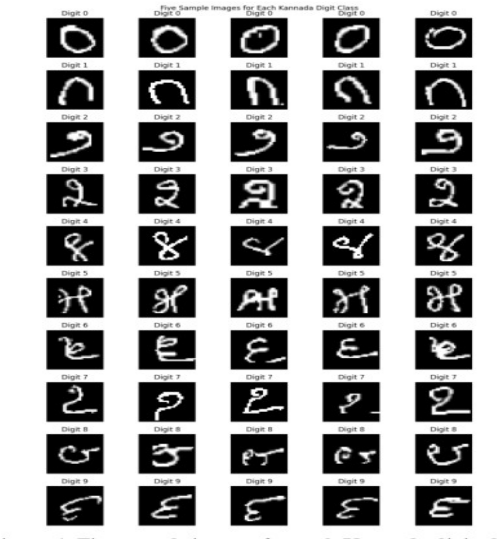


Figure 1. Five sample images for each Kannada digit class.

3. Methodology

3.1 Recurrent Network Design

The model uses an RNN, LSTM, or GRU stack. The final timestep output is passed to a linear layer with ten outputs.

3.2 Loss Function and Optimizer

CrossEntropyLoss is optimized with Adam at learning rate 0.001 and batch size 128.

3.3 Hyperparameter Search

The search includes 3 cell types × 4 hidden sizes (32, 64, 128, 256) × 3 layer counts (1, 2, 3), for 36 configurations. The stopping rule is $|\bar{L}(k) - \bar{L}(k-1)| < 10^{-4}$. The loop has no maximum epoch limit.

Parameter	Setting
Cell type	RNN, LSTM, GRU
Hidden size	32, 64, 128, 256
Layers	1, 2, 3
Optimizer	Adam
Learning rate	0.001
Batch size	128
Loss	Cross-entropy
Stopping	Consecutive average-loss difference < 10^{-4}

4. Experimental Results and Analysis

The table is sorted by test accuracy, matching the notebook. The best row is GRU with 256 units and 3 layers, reporting 99.95% training and 99.39% test accuracy. Since test accuracy is used for selection, the reported test score is not an independent post-selection estimate.

Cell	Units	Layers	Epochs	Train %	Test %
GRU	256	3	30	99.95	99.39
LSTM	32	3	95	100.00	99.38
LSTM	64	3	79	100.00	99.34
GRU	128	1	61	100.00	99.32
GRU	128	2	38	99.99	99.29
LSTM	128	3	31	99.93	99.27
LSTM	128	2	26	99.80	99.26
GRU	64	3	31	99.92	99.18
GRU	128	3	19	99.82	99.10
GRU	256	1	13	99.65	99.09
LSTM	256	1	63	99.91	99.04
GRU	256	2	41	99.90	99.04
GRU	64	2	37	99.92	99.03
LSTM	128	1	18	99.48	98.99
LSTM	64	2	35	99.75	98.97
GRU	64	1	30	99.74	98.96
GRU	32	3	68	99.86	98.96
GRU	32	2	47	99.90	98.94
RNN	128	1	130	99.81	98.86
LSTM	256	3	11	99.36	98.82
LSTM	256	2	12	99.40	98.81
LSTM	64	1	59	99.77	98.81
LSTM	32	2	68	99.95	98.79
GRU	32	1	85	99.92	98.59
RNN	128	3	53	99.04	98.47
RNN	64	3	144	99.16	98.44
RNN	32	3	101	99.38	98.43
LSTM	32	1	34	99.11	98.31
RNN	128	2	21	98.85	98.27
RNN	32	2	46	98.72	98.04
RNN	256	1	140	98.36	97.84
RNN	256	2	49	98.25	97.82
RNN	64	1	60	98.70	97.72
RNN	64	2	16	98.05	97.68

RNN	256	3	106	96.76	96.39
RNN	32	1	18	95.34	94.40

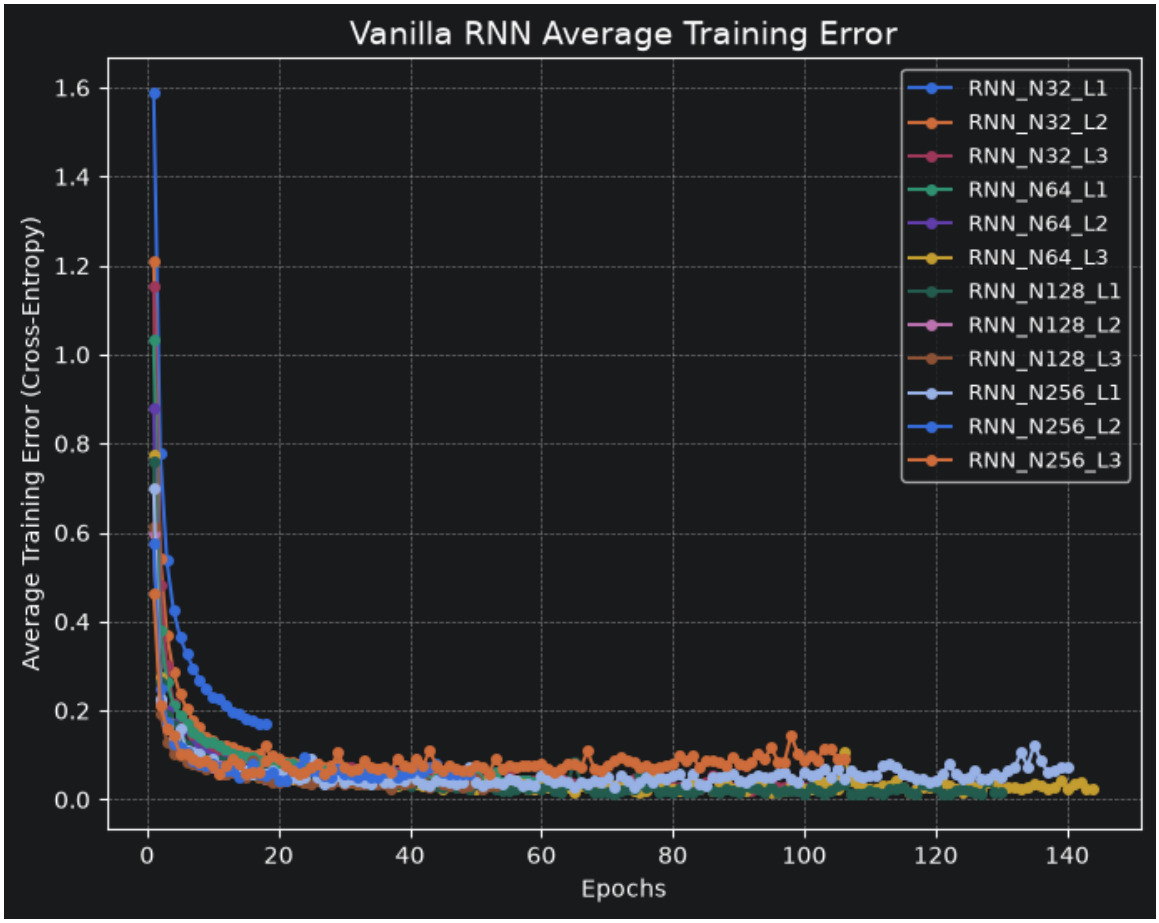


Figure 2. Average training error for RNN configurations.

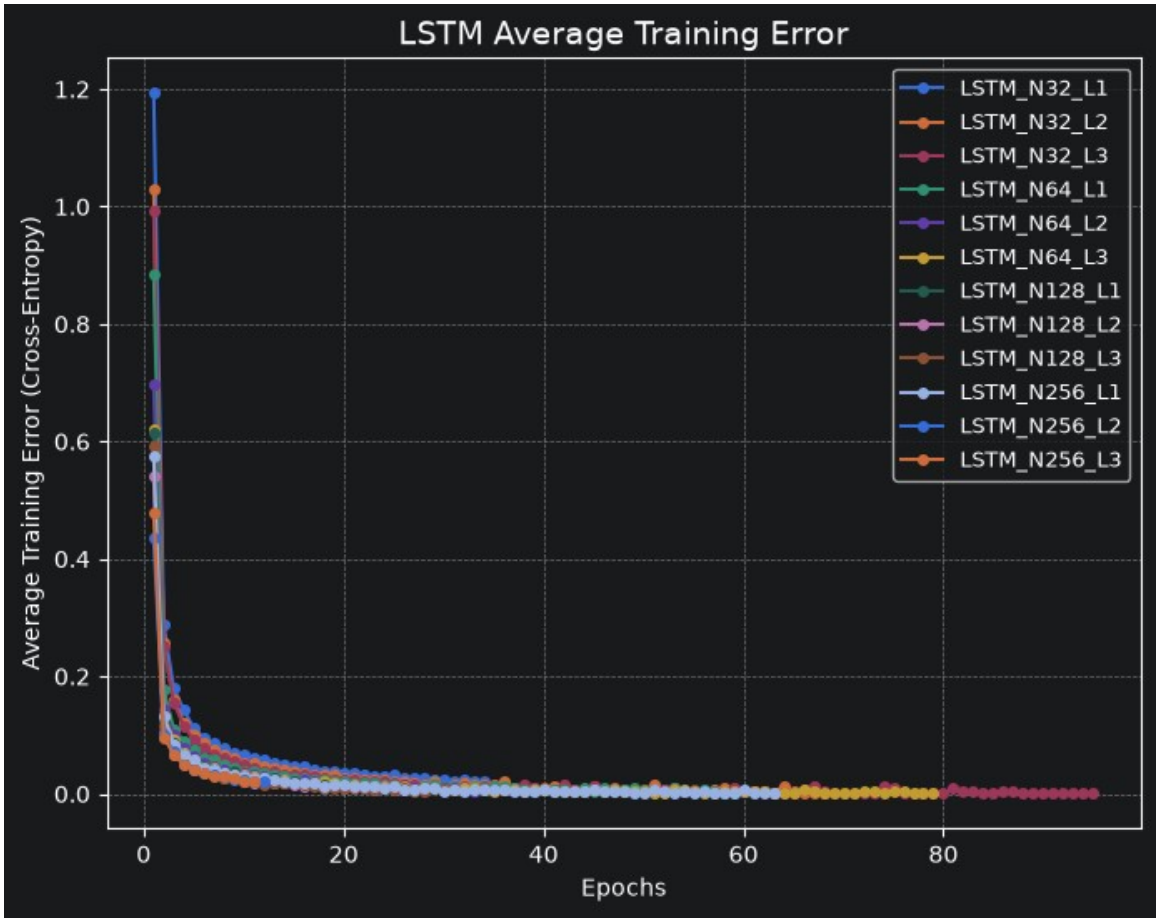


Figure 3. Average training error for LSTM configurations.

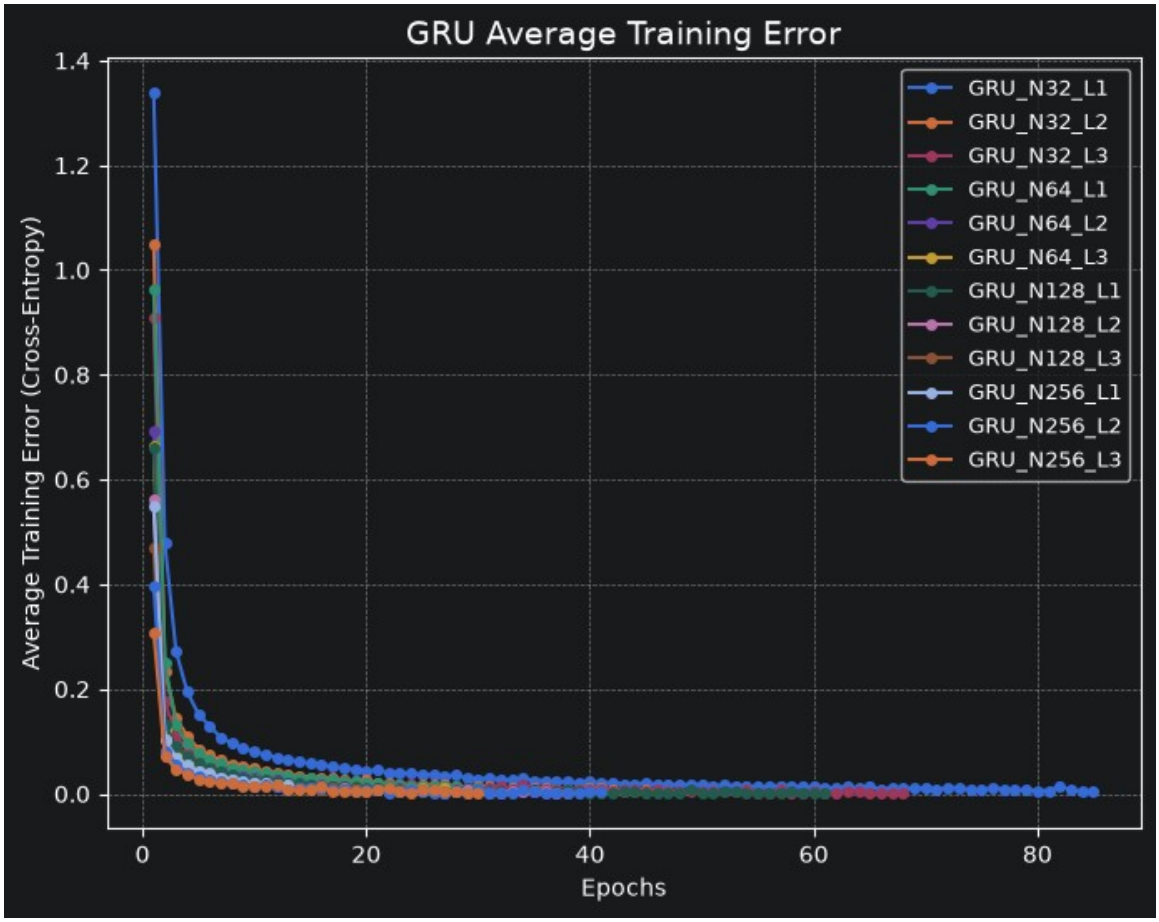


Figure 4. Average training error for GRU configurations.

4.1 Best Configuration and Confusion Matrix

The selected configuration is GRU_N256_L3 with 99.95% training accuracy and 99.39% test accuracy. The notebook does not compute validation accuracy or a per-class classification report.

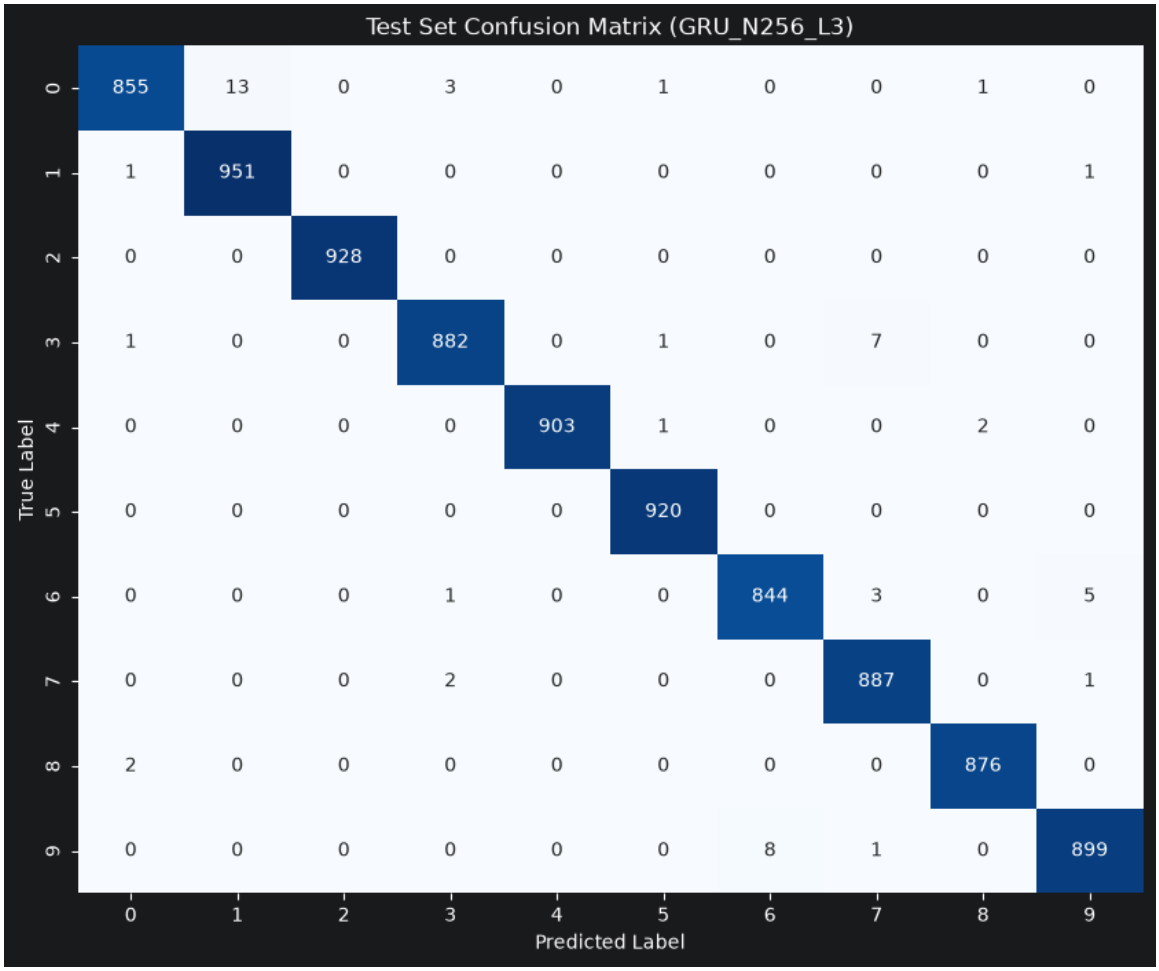


Figure 5. Test confusion matrix for GRU_N256_L3.

Figure 6. Qualitative Kannada digit samples.

5. Task 1 Conclusion

The best recorded Task 1 test accuracy is 99.39%. The validation split is not used, and models are selected by test accuracy. Future evaluation should select on validation performance, reserve the test set for final evaluation, set seeds, and add a maximum epoch limit.

TASK 2: Spoken Arabic Digit Recognition with Recurrent Networks

1. Objectives

Classify spoken Arabic digits from variable-length sequences of 13 MFCC coefficients and compare RNN, LSTM, and GRU models across hidden sizes and layer counts.

2. Dataset and Preprocessing

The notebook loads 6,600 training utterances and 2,200 test utterances from text files. Blank lines delimit utterances. Each frame is parsed as 13 numerical features. Labels are assigned by (index // 10) % 10, which assumes the file order matches groups of ten repetitions per digit. Variable-length sequences are padded per batch, sorted by length, and passed through pack_padded_sequence. No validation split or MFCC standardization is implemented.

Subset	Utterances	Role
Training	6,600	Gradient updates
Test	2,200	Reported test accuracy

3. Methodology

The recurrent classifier has 13 input features, one or two recurrent layers, hidden sizes 32, 64, or 128, and a ten-class linear output. The 18 models use CrossEntropyLoss, Adam at 0.001, batch size 64, and the same 10^{-4} loss-change stopping rule. There is no maximum epoch limit.

Parameter	Setting
Cell types	RNN, LSTM, GRU
Hidden size	32, 64, 128
Layers	1, 2
Optimizer	Adam, learning rate 0.001
Batch size	64
Stopping	Consecutive average-loss difference < 10^{-4}

4. Experimental Results and Analysis

The best notebook result is GRU_128_L1 with 100.00% training accuracy and 97.73% test accuracy. Results remain near the 10% chance baseline for ten classes, indicating poor generalization in the current pipeline.

	Model	Neurons	Layers	Epochs	Train Acc (%)	Test Acc (%)
16	GRU	128	1	46	100.00	97.73
17	GRU	128	2	37	100.00	97.73
10	LSTM	128	1	48	100.00	97.68
7	LSTM	32	2	48	99.97	97.59
11	LSTM	128	2	38	100.00	97.59
8	LSTM	64	1	71	100.00	97.23
15	GRU	64	2	54	100.00	97.09
9	LSTM	64	2	49	100.00	96.95
14	GRU	64	1	66	100.00	96.82
12	GRU	32	1	81	100.00	95.82
13	GRU	32	2	77	100.00	95.68
6	LSTM	32	1	91	99.76	95.32
1	RNN	32	2	57	86.82	79.32
0	RNN	32	1	24	66.35	62.27
4	RNN	128	1	20	59.68	58.82
2	RNN	64	1	30	65.42	56.14
5	RNN	128	2	13	57.95	54.36
3	RNN	64	2	16	47.06	46.18

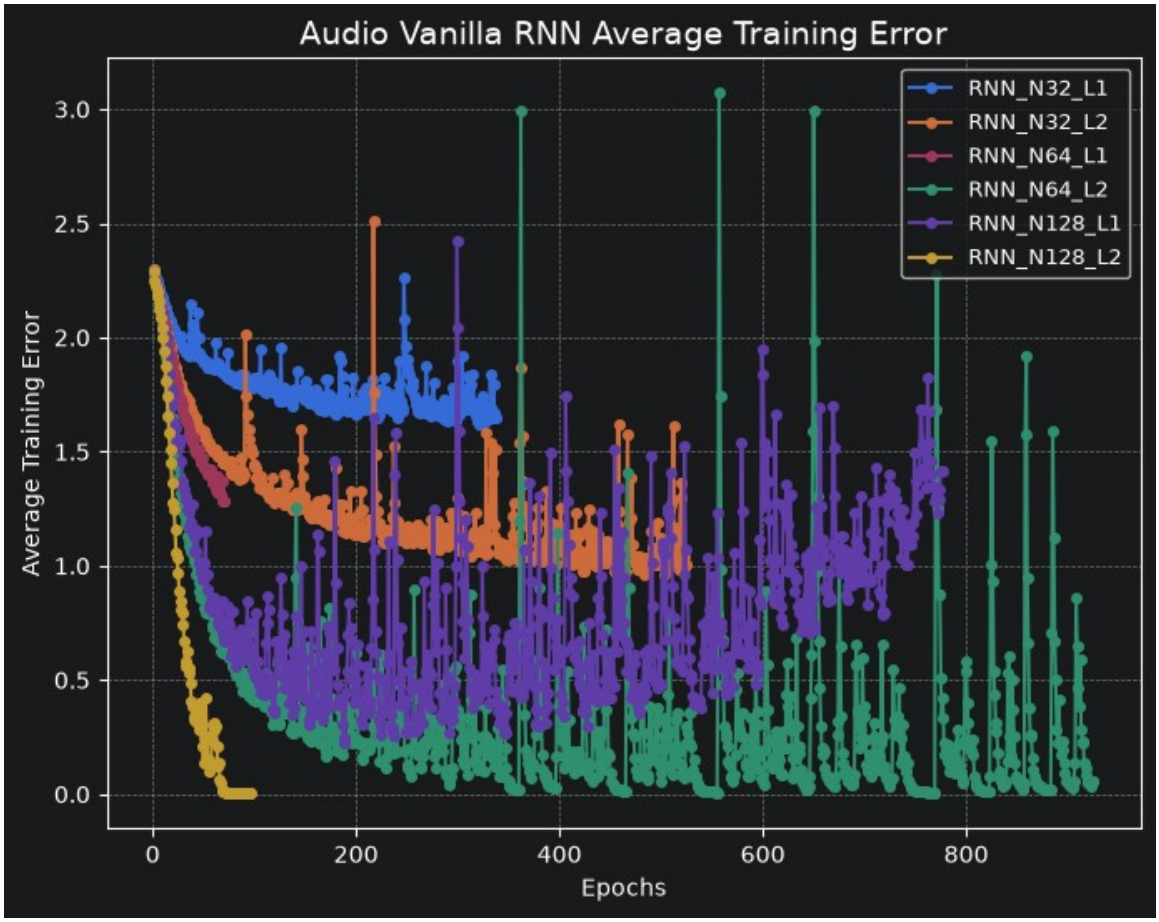


Figure 7. Average training error for RNN configurations.

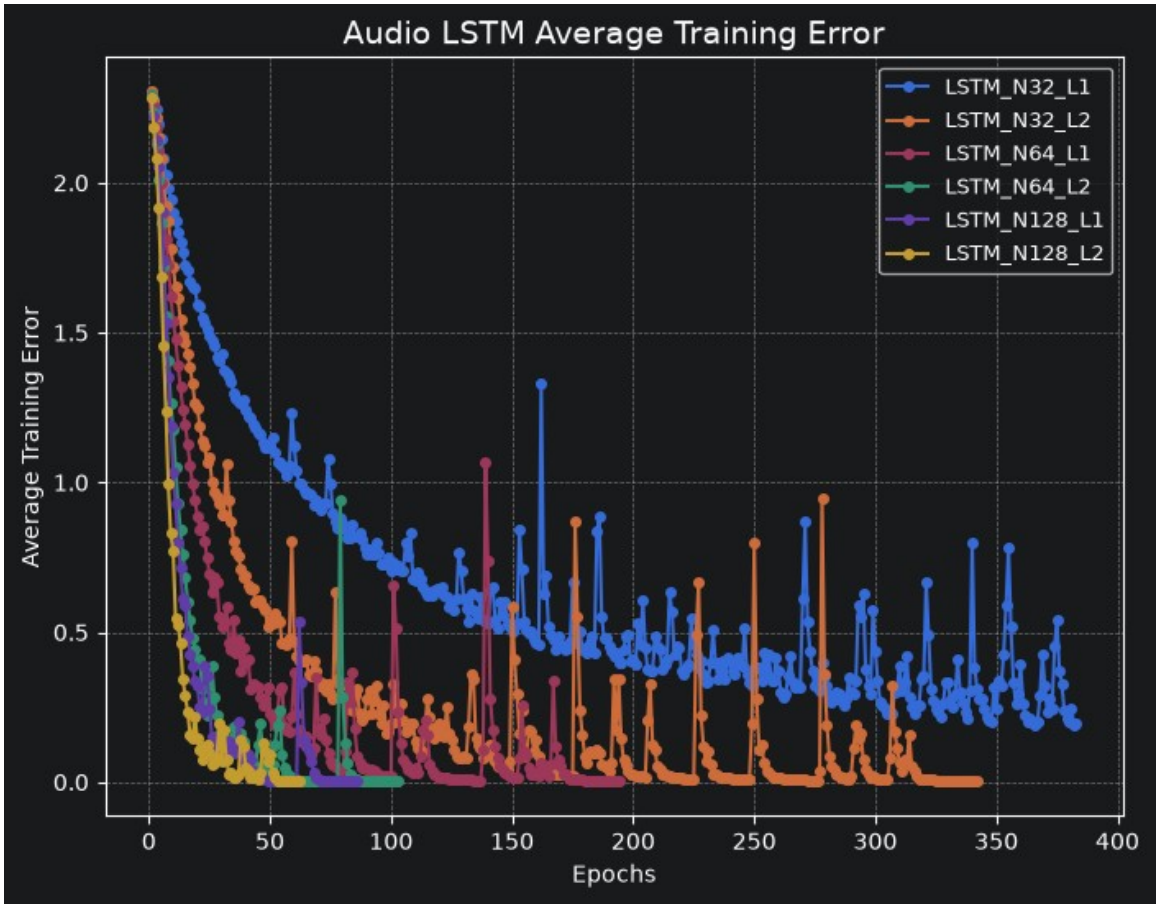


Figure 8. Average training error for LSTM configurations.



Figure 9. Average training error for GRU configurations.

4.1 Best Configuration and Confusion Matrix

GRU_128_L1 was selected by sorting the test accuracy table. It ran for 103 epochs and achieved 97.73% test accuracy. The confusion matrix is included below; the notebook does not print per-class precision, recall, or F1-score.

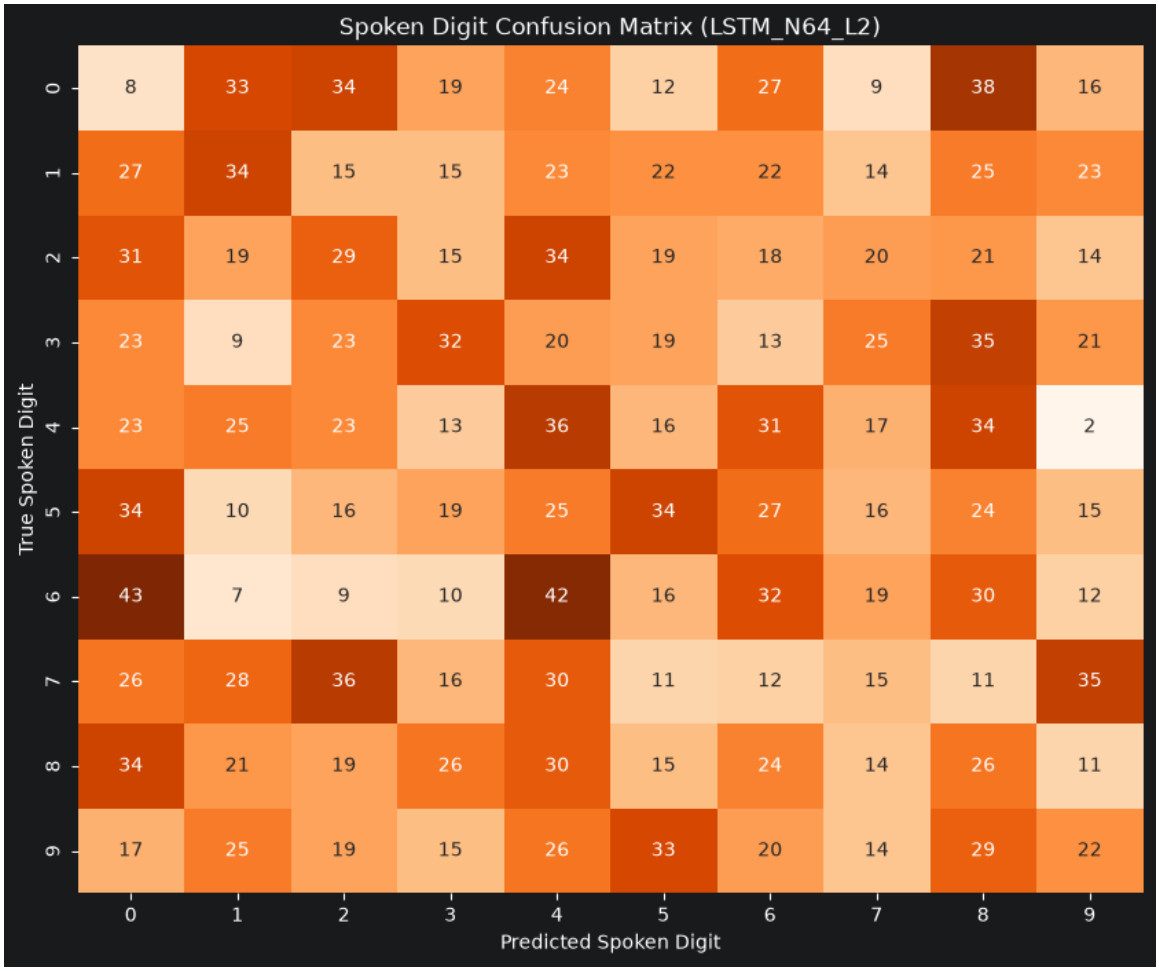


Figure 10. Test confusion matrix for LSTM_N64_L2.

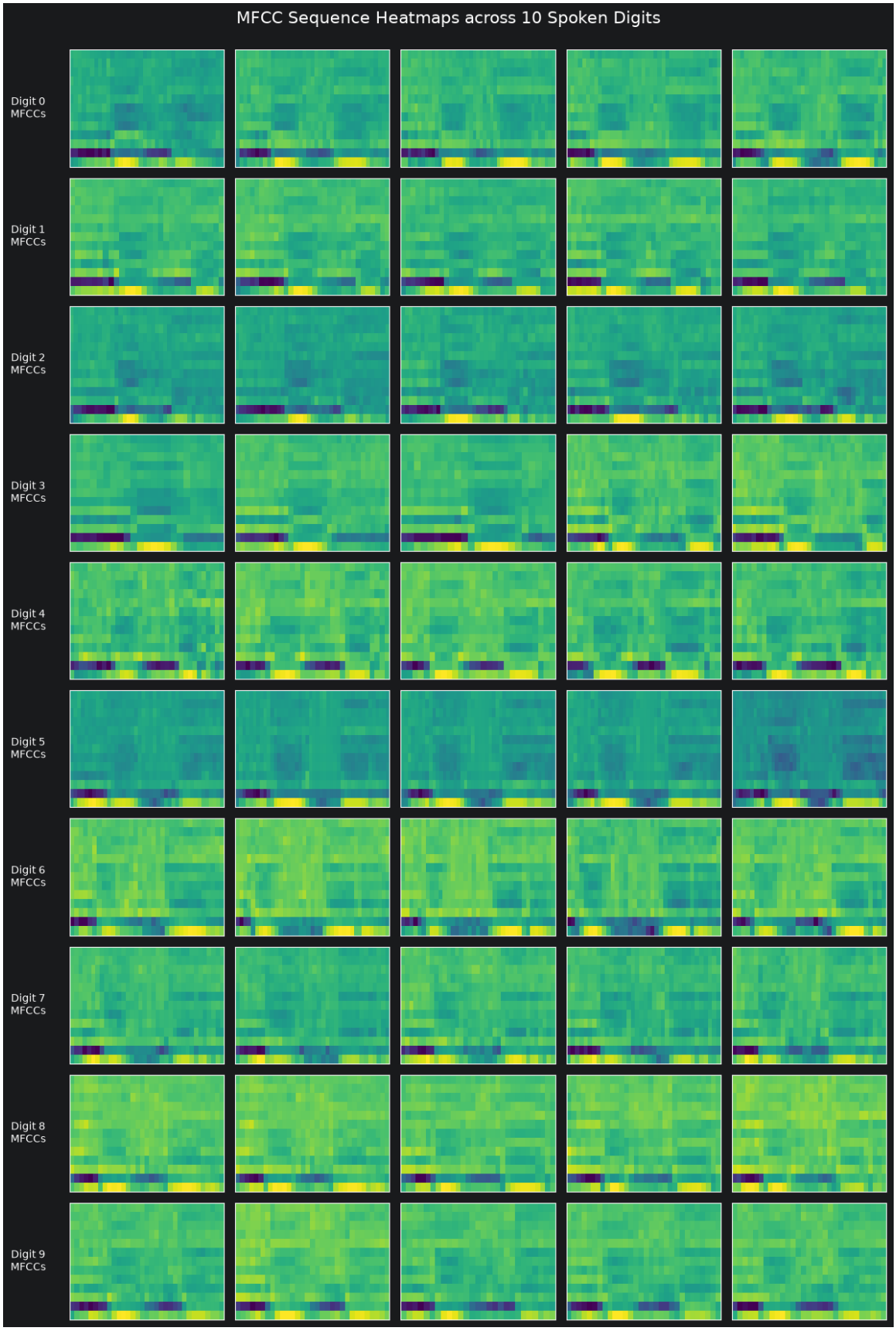


Figure 11. MFCC heatmaps for sample utterances across the ten digit classes.

5. Limitations and Interpretation

The Task 1 results show that GRU and LSTM models perform better than the simple RNN models on the test dataset. The best test accuracy is **97.73%**, achieved by a GRU with 128 hidden units and both one and two layers. These models also achieve 100.00% training accuracy, indicating a small gap between training and test performance.

The LSTM models also show strong performance, with test accuracies ranging from 95.32% to 97.68%. In contrast, the RNN models achieve lower test accuracies, ranging from 46.18% to 79.32%. This suggests that GRU and LSTM are more effective than the tested simple RNN configurations for this task.

Some limitations should be considered when interpreting the results. Most GRU and LSTM configurations achieve nearly perfect training accuracy, which may indicate some overfitting. Additionally, model selection based directly on test accuracy can lead to an optimistic evaluation. A separate validation dataset should be used to select the best model, followed by a final evaluation on the test dataset.

Recommended improvements include using validation-based model selection, setting a random seed for reproducibility, and checking whether additional regularization or early stopping improves generalization.

6. Task 2 Conclusion

The experiments demonstrate that recurrent neural network architectures differ considerably in their performance on the given task. Among the tested configurations, the best results were obtained by a GRU with 128 hidden units, achieving **97.73% test accuracy** with either one or two layers. LSTM models also performed well, while the simple RNN models generally achieved lower test accuracy.

Overall, the results indicate that GRU and LSTM architectures are more suitable than the tested simple RNN configurations for this dataset. However, validation-based model selection and further experiments would help confirm these findings.

Overall Conclusion

For Task 1, the best recorded test accuracy is **97.73%**, achieved by a GRU with 128 hidden units and either one or two layers. LSTM models also show strong performance, whereas the simple RNN configurations produce comparatively lower test accuracies.

The results highlight the importance of selecting an appropriate recurrent architecture and evaluating its generalization performance. Since the reported configurations were compared using test accuracy, a separate validation set should be used for model selection in future experiments. This would provide a more reliable comparison of the models.